
Five Nights at Dana Design Manual

Introduction

Five Nights at Dana is a survival point-and-click style game implemented in Java. In the game, you play as a computer science instructor working late nights in their office inside Dana Engineering at Bucknell University. Throughout the night, the player must defend themselves from students roaming the building looking for late-night help with assignments and projects. To survive until 6 a.m., the player must strategically close doors, stop elevators, seal vents, monitor cameras, and manage other building systems to prevent students from reaching the office.

From a technical standpoint, the system is divided into seven major sections: AI, Controllers, Core, Managers, Rendering, Systems, and Tools. Each package contains code responsible for a specific portion of the game architecture. The AI package contains the logic that governs student movement and behavior throughout the building. The Controllers package manages JavaFX screens and user interface flow, including the Main Menu, Camera View, Game View, and Jumpscare sequences. The Core package ties the project together by running the main game loop and managing session state. Managers act as centralized coordinators for systems throughout the game; for example, the StudentManager updates all active student AI and coordinates movement behavior. Rendering contains the code responsible for displaying and mapping in-game camera feeds. Systems implement gameplay mechanics such as elevators, stairwells, classrooms, doors, and vents. Finally, the Tools package contains simulation and analytics utilities that were used during development to test gameplay balance and analyze common student movement patterns.

Our codebase follows a modular, layered structure where each package has a clearly defined responsibility, making separation of concerns a central design feature. The AI layer models student behavior and movement logic, Managers coordinate shared state and cross-system rules, Controllers handle user input and UI flow, Systems implement gameplay mechanics, and Core drives the game loop and session lifecycle. This organization keeps logic isolated, reduces coupling, and makes the project easier to test, debug, and extend because changes within one subsystem are less likely to break another.

In addition to the gameplay implementation itself, the project also contains extensive testing suites to ensure that systems behave correctly under different gameplay conditions. This modular organization allowed our team to develop gameplay systems independently while still maintaining strong integration between packages.

User Stories

To guide our design decisions, we developed user stories representing different player archetypes. These user stories helped shape the mechanics, pacing, and replayability of the game.

George Thompson: Completionist

Quote:

"I won't stop until I've unlocked every ending, every easter egg, every secret room."

Narrative:

George wants to experience every feature the game has to offer. He explores hidden mechanics, grinds alternate game modes, and searches for secret interactions and Easter eggs. Unlockables and hidden content motivate him to replay the game long after completing the main experience.

Jen Armstrong: Game Design Appreciator

Quote:

"I enjoy games with good story, art, and mechanics."

Narrative:

Jen appreciates the craftsmanship behind game development. Rather than simply playing the game, she studies how systems interact and why certain mechanics feel satisfying. She values games with carefully designed mechanics and strong thematic consistency.

Oya Bakırcıoğlu: Casual Gamer

Quote:

"I just want to experience the atmosphere without feeling completely overwhelmed."

Narrative:

Oya is newer to horror games and prefers experiences with gradual learning curves. She values tutorials, readable mechanics, and adjustable difficulty settings that allow her to improve over time without becoming frustrated.

Rodrigo Sánchez: Lore Hunter

Quote:

"I want to understand every secret this game is hiding."

Narrative:

Rodrigo enjoys uncovering hidden details and piecing together environmental storytelling. He carefully observes movement patterns, hidden mechanics, and subtle clues embedded throughout the game world.

Object Oriented Design and User Story Implementations

To properly address object-oriented design within the game, our team began development by creating a comprehensive UML diagram. This diagram allowed us to visualize how classes and systems interacted while ensuring responsibilities were appropriately separated across the project architecture.

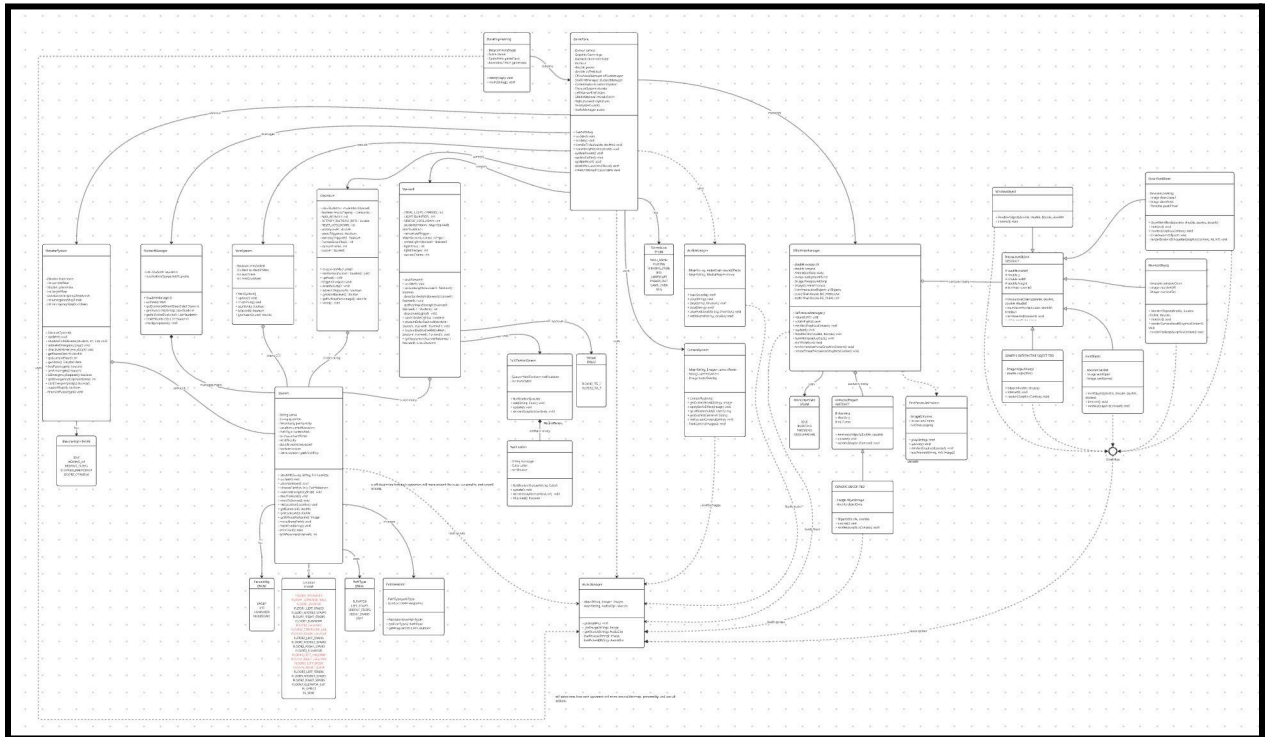


Figure 1: Overall UML diagram of the system

Single Responsibility Principle

One of the most important object-oriented principles implemented throughout the project was the Single Responsibility Principle. This principle states that classes and methods should have one clear, well-defined purpose. By following this approach, the system became easier to understand, maintain, and expand.

This design philosophy is especially visible within the Systems package. Rather than placing all gameplay mechanics into one monolithic file, individual mechanics were separated into dedicated classes such as ElevatorSystem, ClassroomSystem, StairwellSystem, and VentSystem. Each class contains methods responsible for one isolated gameplay feature, reducing complexity and improving modularity.

This organization directly supports Jen Armstrong’s user story as the Game Design Appreciator. By separating gameplay systems into isolated components, the game mechanics become more polished, expandable, and internally consistent.

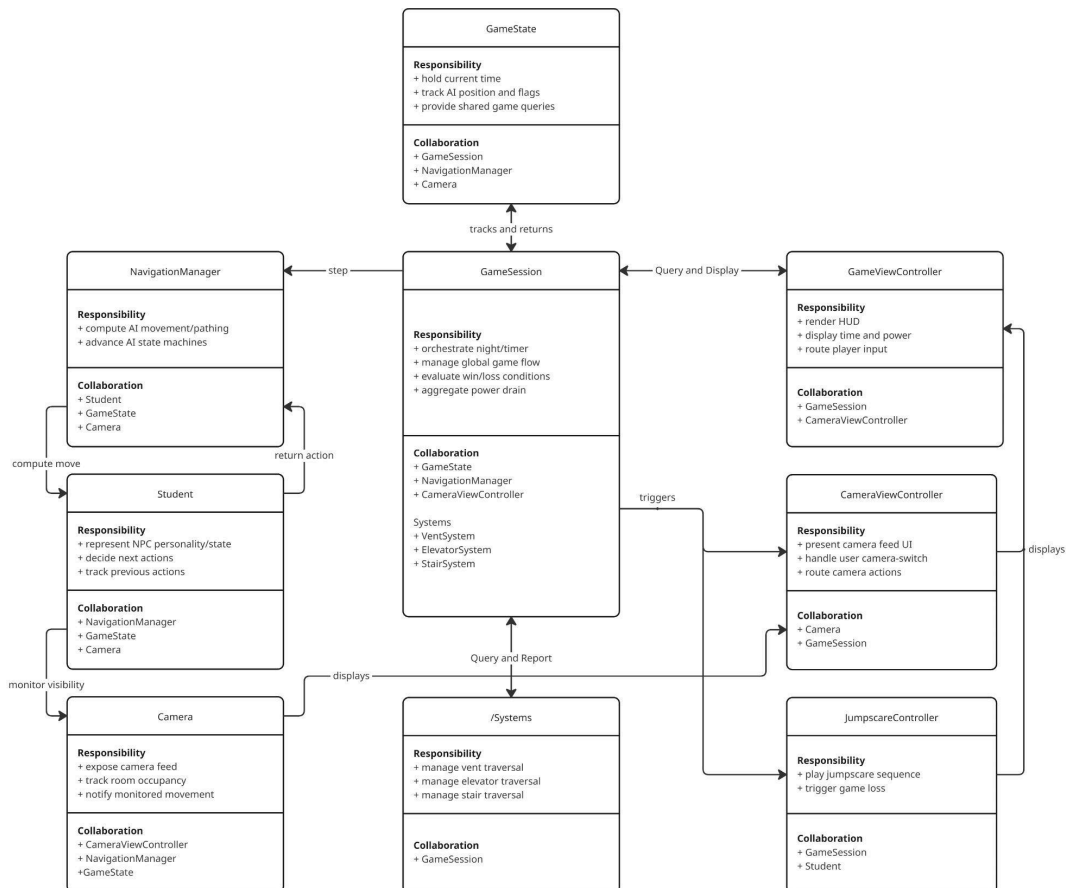


Figure 2: CRC card representation of core files

Encapsulation and Abstraction Through Enumerations

Another major object-oriented principle implemented in the project was encapsulation and abstraction through the use of enumerations. Several important systems rely on enums to simplify logic while improving readability and maintainability.

The Student class uses three particularly important enumerations:

- Personality
- Location
- PathType

Each enumeration represents a different abstraction layer of the AI system. Personality determines how a student behaves, PathType defines preferred movement methods, and Location represents all possible spatial nodes throughout the building.

Using enums provides several advantages, such as compile-time safety, improved readability, cleaner switching logic, easier balancing and expansion, and reduced reliance on fragile string-based comparisons.

The Student AI system addresses Rodrigo Sánchez's user story as the Lore Hunter by creating varied and unpredictable movement patterns between playthroughs. Because students follow different personalities and movement preferences, players can study behavior patterns while still encountering emergent variation.

In addition, the Student class stores the game's difficulty level using an AI-level system ranging from 0–20. This implementation addresses Oya Bakırcıoğlu's user story by allowing difficulty scaling that gradually introduces pressure while remaining accessible to new players.

Location Enum

The Location enum models all traversable spaces within the game. The building is represented as a three-floor graph containing approximately forty nodes.

Examples include:

- FLOOR1_ENTRANCE
- FLOOR1_LOUNGE
- FLOOR2_CLASSROOM
- FLOOR3_TESTING_LAB
- FLOOR3_AT_DOOR
- IN_OFFICE

Special transitional states such as IN_VENT and IN_ELEVATOR are also included.

The enum-based implementation ensures type safety while simplifying pathfinding and gameplay logic.

Personality Enum

The Personality enum defines five distinct AI archetypes:

Personality	Behavior
SHY	Prefers vents and stalls when watched
EAGER	Aggressively prefers elevators
CONFUSED	Randomized and erratic movement
PERSISTENT	Slowly pressures the player
RUNNER	Extremely fast sprint-based attacker

Each personality defines movement opportunity intervals, path preferences, observation behavior and special gameplay mechanics. For example, SHY students freeze when observed through cameras, RUNNER students charge before sprinting directly toward the office, and PERSISTENT students linger at the door before attacking. This create recognizable personalities that players can learn and counter.

PathType Enum

The PathType enum defines methods of movement throughout the building:

- NORMAL
- VENT
- LEFT_STAIRS
- MIDDLE_STAIRS
- RIGHT_STAIRS
- ELEVATOR

Each path type interacts differently with player defenses. For example:

- Vents can be sealed
- Elevators can be stopped
- Stairwells contain unique gameplay mechanics

This system creates tactical variety and encourages active resource management.

Student Class Architecture

The Student class represents an autonomous AI-controlled entity attempting to reach the player's office.

The class encapsulates:

- AI state
- Movement timing
- Personality behavior
- Recent-location memory
- Difficulty scaling

The memory system stores recent locations inside a FIFO deque. This memory is used to discourage repetitive movement loops and allows gameplay systems to rewind student progress through pushback mechanics.

Movement Opportunity System

The movement system is heavily inspired by the original Five Nights at Freddy's games. Students do not move continuously every frame. Instead, they receive discrete movement opportunities.

The process works as follows:

1. Reset movement timer
2. Count down each frame
3. When timer expires:
 - Roll against AI level
 - If successful, perform movement step
 - Reset timer

Movement intervals are personality-dependent and scale based on AI difficulty.

The probabilistic movement check uses:

$$\text{roll} \leq \text{aiLevel}$$

Where AI level ranges from 0–20:

- 0 = never moves
- 20 = always moves

This system creates suspense because movement is uncertain even under identical gameplay conditions.

Design Philosophy

The overall AI and gameplay design prioritizes:

- Replayability
- Emergent behavior
- Predictable personalities
- Modular architecture
- Tactical gameplay depth

Our AI system was designed so that players can gradually learn broad behavioral patterns without ever fully predicting individual movements. This balance between consistency and randomness keeps gameplay tense while rewarding player learning and experimentation.

Additionally, the layered project architecture allowed our team to independently develop systems while maintaining clean integration between gameplay mechanics, rendering, UI, and AI logic.

Conclusion

Overall, Five Nights at Dana demonstrates multiple object-oriented design principles, including encapsulation, abstraction, separation of concerns, and modular system design. The project combines probabilistic AI systems, graph-based navigation, modular gameplay mechanics, and centralized management architecture to create a replayable and technically sophisticated survival horror experience.

By implementing layered systems and personality-driven AI, we were able to create gameplay that appeals to multiple player archetypes, ranging from casual players to completionists and systems-focused players. The result is a game that is both technically robust and engaging from a gameplay perspective.